

TASK 1

DATA CLEANING AND PREPROCESSING

Student Name	Arshiya Chaudhary
Course	B.Sc. Data Science
Tools Used	Python, Pandas, Excel
Deliverables	Cleaned dataset and summary of changes

1. Objective

The objective of this task is to clean and preprocess a raw dataset containing common data-quality problems such as missing values, duplicate records, inconsistent text, mixed date formats, unclear column names, and incorrect data types. The cleaned data should be consistent, reliable, and ready for analysis or visualization.

2. Tools and Dataset

Tools: Python & Pandas.

Suggested Kaggle datasets Sales Data. For this report, the workflow is written generically so it can be applied to any of these datasets. The exact changes and row counts should be recorded after running the workflow on the selected file.

3. Data Cleaning Workflow

1. **Load the dataset:** Import the CSV or Excel file and inspect its shape, column names, sample rows, and data types.
2. **Inspect missing values:** Count null values in each column and decide whether to remove or fill them based on the meaning and amount of missing data.
3. **Remove duplicates:** Identify duplicate rows and remove only records that are genuinely repeated.
4. **Standardize text:** Remove extra spaces, use consistent capitalization, and map equivalent labels (for example, 'f', 'Female', and 'female') to one value.
5. **Standardize dates:** Parse dates into a consistent datetime format and investigate values that cannot be parsed.
6. **Clean column names:** Use lowercase names, replace spaces with underscores, and remove unnecessary punctuation.
7. **Correct data types:** Convert numeric columns to numeric types, dates to datetime, and categories to suitable text/category types.
8. **Check unusual values:** Review impossible values and potential outliers; correct only when supported by source information or a documented rule.
9. **Validate and export:** Recheck nulls, duplicates, types, and value ranges, then save the cleaned dataset and a change log.

4. Example Python Code

Update the file name and column names to match the dataset you choose.

```
import pandas as pd

# 1. Load data
df = pd.read_excel("Online-Store-Orders.xlsx", sheet_name="Sheet1")

# 2. Inspect the dataset
print("Shape:", df.shape)
print(df.head())
print(df.info())
print("Missing values:\n", df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 3. Standardize column headers to snake_case
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(r"^[a-z0-9]+", "_", regex=True)
    .str.strip("_")
)

# 4. Remove exact duplicate rows and duplicate Order IDs
df = df.drop_duplicates()
if "orderid" in df.columns:
    df = df.drop_duplicates(subset=["orderid"])

# 5. Standardize text columns (trim extra spaces)
for col in df.select_dtypes(include=["object"]).columns:
    df[col] = df[col].astype(str).str.strip()

# Standardize categorical values (title case for names/status)
for col in ["product", "paymentmethod", "orderstatus", "referralsource"]:
    if col in df.columns:
        df[col] = df[col].str.title()

# 6. Convert date columns, if present
if "date" in df.columns:
    df["date"] = pd.to_datetime(df["date"], errors="coerce")

# 7. Convert numeric columns, if present
numeric_int_cols = ["quantity", "itemsincart"]
for col in numeric_int_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce").astype("Int64")

numeric_float_cols = ["unitprice", "totalprice"]
for col in numeric_float_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce").round(2)

# 8. Handle missing values
# 'couponcode' is null when no coupon was applied
if "couponcode" in df.columns:
    df["couponcode"] = df["couponcode"].replace("nan", None).fillna("None")
```

```

# 8. Handle missing values
# 'couponcode' is null when no coupon was applied
if "couponcode" in df.columns:
    df["couponcode"] = df["couponcode"].replace("nan", None).fillna("None")

# 9. Validate business logic: ensure totalprice equals quantity * unitprice
if {"quantity", "unitprice", "totalprice"}.issubset(df.columns):
    expected_total = (df["quantity"] * df["unitprice"]).round(2)
    df["totalprice"] = df["totalprice"].fillna(expected_total)

# 10. Review quality after cleaning
print("Final shape:", df.shape)
print("Missing values after cleaning:\n", df.isnull().sum())
print("Duplicate rows after cleaning:", df.duplicated().sum())
print(df.dtypes)

# 11. Export cleaned data
df.to_excel("cleaned_online_store_orders.xlsx", index=False)
df.to_csv("cleaned_online_store_orders.csv", index=False)

```

[3]
✓ Os

```

from google.colab import files

# Export to CSV / Excel first
df.to_csv("cleaned_online_store_orders.csv", index=False)
df.to_excel("cleaned_online_store_orders.xlsx", index=False)

# Trigger browser download
files.download("cleaned_online_store_orders.csv")
# files.download("cleaned_online_store_orders.xlsx")

```

▼

```

...
orderid      0
date         0
customerid   0
product      0
quantity     0
unitprice    0
shippingaddress 0
paymentmethod 0
orderstatus  0
trackingnumber 0
itemsincart   0
couponcode    0
referralsource 0
totalprice    0
dtype: int64
Duplicate rows after cleaning: 0
orderid      object
date         datetime64[ns]
customerid   object
product      object
quantity     Int64
unitprice    float64
shippingaddress  object
paymentmethod  object
orderstatus  object
trackingnumber  object
itemsincart   Int64
couponcode    object
referralsource  object
totalprice    float64

```

```

dtypes: datetime64[ns](1), float64(2), int64(2), object(9)
memory usage: 131.4+ KB
None
Missing values:
  OrderID      0
  Date         0
  CustomerID   0
  Product      0
  Quantity     0
  UnitPrice    0
  ShippingAddress 0
  PaymentMethod 0
  OrderStatus  0
  TrackingNumber 0
  ItemsInCart  0
  CouponCode   309
  ReferralSource 0
  TotalPrice   0
dtype: int64
Duplicate rows: 0
Final shape: (1200, 14)
Missing values after cleaning:
  orderid      0
  date         0
  customerid   0
  product      0
  quantity     0
  unitprice    0
  shippingaddress 0
  paymentmethod 0

```

```

...
  CouponCode ReferralSource TotalPrice
0  SAVE10      Instagram    2853.10
1  SAVE10      Referral     302.70
2  FREESHIP     Email      2753.40
3  SAVE10      Facebook    273.19
4  SAVE10      Email      2504.04
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OrderID               1200 non-null  object
1   Date                  1200 non-null  datetime64[ns]
2   CustomerID            1200 non-null  object
3   Product               1200 non-null  object
4   Quantity              1200 non-null  int64
5   UnitPrice             1200 non-null  float64
6   ShippingAddress        1200 non-null  object
7   PaymentMethod         1200 non-null  object
8   OrderStatus           1200 non-null  object
9   TrackingNumber        1200 non-null  object
10  ItemsInCart           1200 non-null  int64
11  CouponCode            891 non-null   object
12  ReferralSource        1200 non-null  object
13  TotalPrice            1200 non-null  float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(9)
memory usage: 131.4+ KB

```

```

... Shape: (1200, 14)
   OrderID      Date CustomerID Product  Quantity  UnitPrice  \
0  ORD200000  2023-01-04    C72649  Monitor         5     570.62
1  ORD200001  2024-08-23    C75739   Phone         2     151.35
2  ORD200002  2024-02-27    C81728  Tablet         5     550.68
3  ORD200003  2023-10-15    C33540   Chair         1     273.19
4  ORD200004  2025-05-08    C81840  Printer         4     626.01

   ShippingAddress PaymentMethod OrderStatus TrackingNumber  ItemsInCart  \
0      928 Main St      Debit Card      Shipped    TRK37947903           7
1      823 Main St           Online      Shipped    TRK91186779           3
2      512 Main St      Credit Card  Cancelled    TRK42903982           8
3      275 Main St      Debit Card      Returned    TRK62788070           5
4      668 Main St           Online  Delivered    TRK29241424           8

   CouponCode ReferralSource  TotalPrice
0      SAVE10      Instagram    2853.10
1      SAVE10      Referral     302.70
2  FREESHIP           Email    2753.40
3      SAVE10      Facebook     273.19
4      SAVE10           Email    2504.04
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   OrderID             1200 non-null  object
1   Date                1200 non-null  datetime64[ns]
...
CustomerID           object
product              object
quantity             Int64
unitprice            float64
shippingaddress       object
paymentmethod         object
orderstatus          object
trackingnumber        object
itemsincart          Int64
couponcode           object
referralsource        object
totalprice           float64
dtype: object

```

6. Conclusion

Data cleaning is an essential step before analysis because missing, duplicated, inconsistent, or incorrectly typed data can lead to misleading summaries and visualizations. By inspecting the dataset, applying justified cleaning rules, validating the results, and documenting each change, the dataset becomes more consistent and suitable for further analysis. The final report should include the cleaned file and the actual changes observed.

7. Viva / Interview Questions and Answers

What are missing values and how do you handle them?

Missing values are fields for which no value is recorded, often represented as NaN or blank cells. I first measure them using `df.isnull().sum()`, then decide whether to remove rows/columns or impute values using a suitable statistic or category. The decision depends on the amount of missing data and the business meaning of the column.

How do you treat duplicate records?

I check duplicates with `df.duplicated().sum()`. I use `df.drop_duplicates()` to remove exact duplicate rows. If duplicates are defined by particular fields, I use `subset=`, for example `df.drop_duplicates(subset=['customer_id'])`. I verify that repeated records are not valid transactions before deleting them.

What is the difference between `dropna()` and `fillna()` in Pandas?

`dropna()` removes rows or columns containing missing values, depending on its arguments. `fillna()` replaces missing values with a specified value or method, such as a median or category. Dropping can reduce the dataset, while filling preserves records but may introduce assumptions.

What is outlier treatment and why is it important?

Outlier treatment means identifying unusually high or low observations and deciding how to handle them. Outliers may be genuine or caused by data-entry errors. They can affect averages and some models, so I investigate them using domain rules, box plots, IQR, or z-scores before correcting, capping, transforming, or retaining them.

Explain the process of standardizing data.

Standardizing data means making values consistent in format and meaning. Examples include converting 'F', 'female', and 'Female' to one label, removing extra spaces, using consistent date formats, and applying consistent units. Numeric standardization can also mean scaling values, which is a separate preprocessing step.

How do you handle inconsistent data formats such as date/time?

I inspect the original values and determine the expected format. In Pandas, `pd.to_datetime(column, errors='coerce', dayfirst=True)` can parse dates where appropriate. I review values converted to NaT because they may be invalid or use a different format, and then store dates as datetime values.

What are common data cleaning challenges?

Common challenges include missing or incorrect values, duplicates, inconsistent labels, mixed date formats, wrong data types, outliers, spelling differences, encoding problems, and unclear business rules. Another challenge is avoiding the removal of valid records while correcting errors.

How can you check data quality?

I check completeness (missing values), uniqueness (duplicates), validity (allowed ranges and formats), consistency (matching labels and units), accuracy where source verification is possible, and data types. I compare row counts before and after cleaning and document all changes.

9. Useful Pandas Commands

Command	Purpose
<code>df.head()</code>	View the first five rows.
<code>df.shape</code>	Check number of rows and columns.

df.info()	Inspect column types and non-null counts.
df.isnull().sum()	Count missing values in each column.
df.duplicated().sum()	Count duplicate rows.
df.drop_duplicates()	Remove duplicate rows.
df.dropna()	Remove rows/columns with missing values.
df.fillna(value)	Replace missing values.
df.dtypes	Inspect data types.
df.to_csv('cleaned_dataset.csv', index=False)	Save the cleaned dataset.